CSci 23: Module 1 →

# Back-end Web Dev

*

Version: v3.2026

Rodney Maniego Jr.
Department of Computer Science and Technology
Faculty of Computing * Visayas State University

**Disclaimer:** This presentation is specially prepared as part of the **CSci 23 Applications Development and Emerging Technologies** course for the **Visayas State University**. It is provided for educational purposes under fair use. Unauthorized copying, distribution, or reproduction of this material, in whole or in part, is strictly prohibited.

*

# Course Outcome

Deploy websites using frameworks and platforms.

# Learning Outcomes

1. Identify fundamental concepts on web development.
2. Develop backend server using frameworks.
3. Deploy backend server using Platform- as-a-Service (PaaS).

# Outline

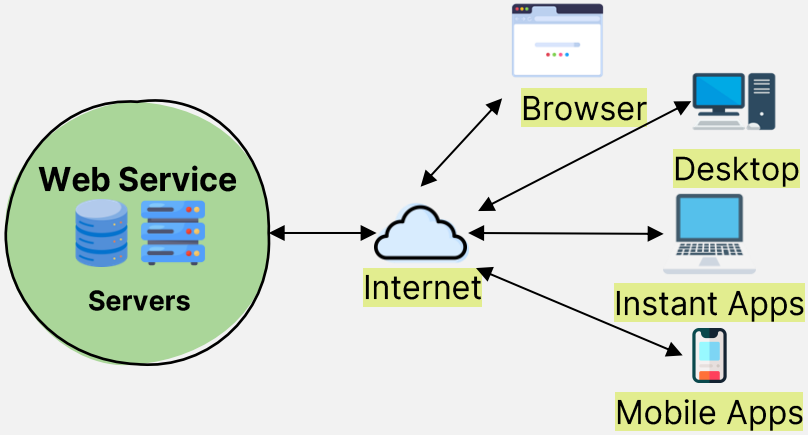
1. Python
2. Python Projects
3. Package Manager
4. Third-Party Libraries

Back-end Web Dev

# Python

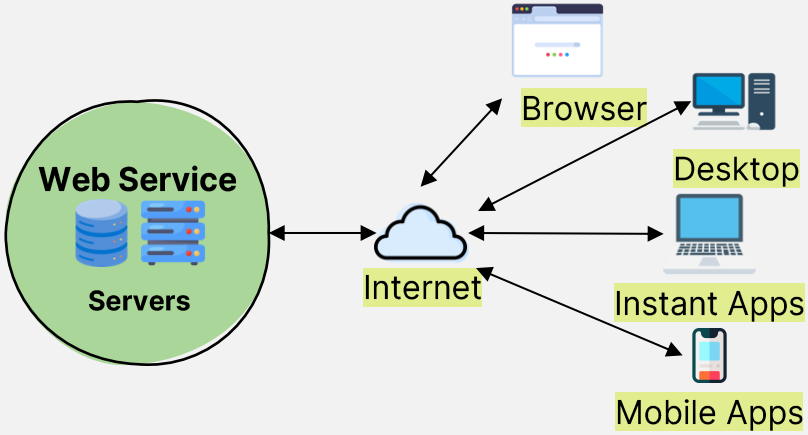
# What is a Server?

It is an application or device that **provides services** or resources to clients.



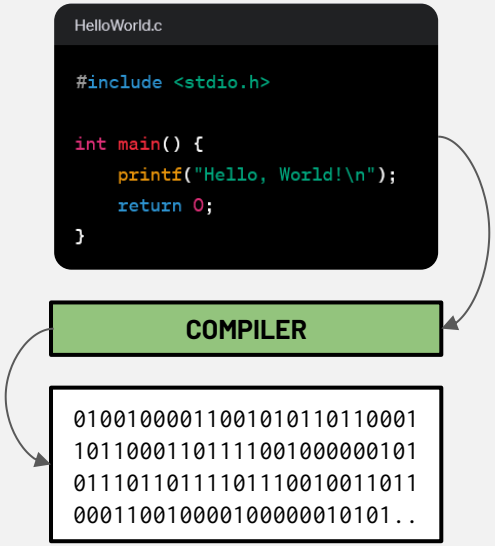
# What is Backend-end?

It is a **server-side application layer** responsible for business logic, data processing, and database interaction.



# Compiler

It is a program that translates code written in a high-level programming language into a machine-readable format that a computer can execute.



# Interpreter

It is a program that executes high-level code by translating it into machine code, line-by-line.

# What is Python?

Python is an **interpreted**, object-oriented, high-level programming language with dynamic semantics.

ABC

Python 1985

Python 0.9 1991

Python 2.0 2000

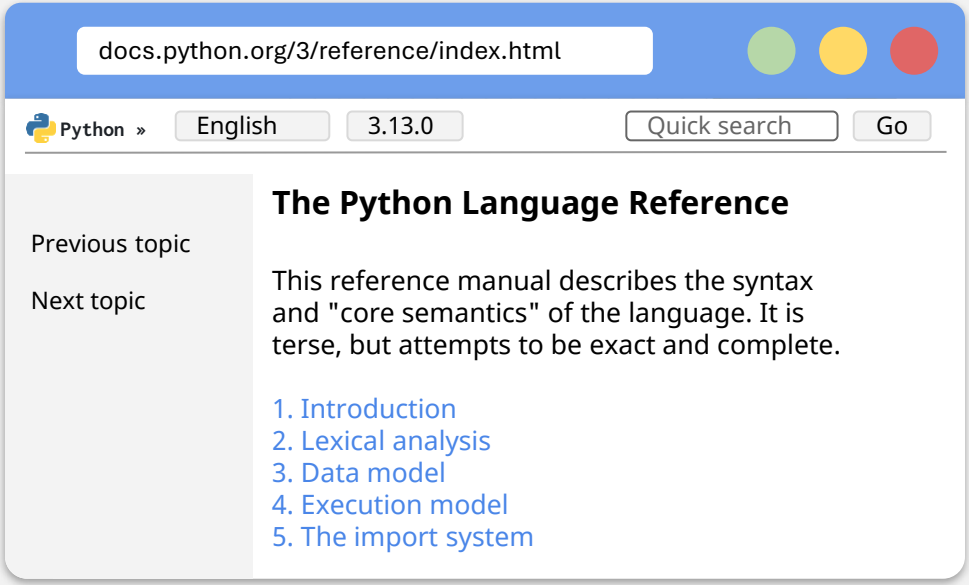
Python 3.0 2008

Python 3.12 2024

**Guido van Rossum**

2020
Python 2.7
(Discontinued)

# What is Python?




**NOTE:** Contents might be too technical for new programmers.

# Why Python?

Multipurpose

Extensive Libraries

Supportive Community

AI and Data Science

Simple and Readable

Highly Extensible

Mature Language

# Why Python?

hello_world.py

```python
print("Hello World!")
```

# Zen of Python

1. Beautiful is better than ugly
2. Explicit is better than implicit
3. Simple is better than complex
4. Complex is better than complicated
5. Readability counts

https://peps.python.org/pep-0020/

# Zen of Python

docs.python.org/3/library/index.html

**Python Enhancement Proposals** | Python » PEP Index » PEP 8

**Contents**


- Introduction
- Code Lay-out
- Comments
- Naming Conventions
- Programming Recommendations
- References
- Copyright


## PEP 8 – Style Guide for Python Code

**Author:** Guido van Rossum, Barry Warsaw, Nick Coghlan
**Status:** Active
**Type:** Process
**Created:** 05-Jul-2001
**Post-History:** 05-Jul-2001, 01-Aug-2013

### Introduction

This document gives coding conventions for the Python code comprising the standard library in the main Python distribution.

**NOTE:** Contains the best practices in coding with Python.

Back-end Web Dev

# Python Projects

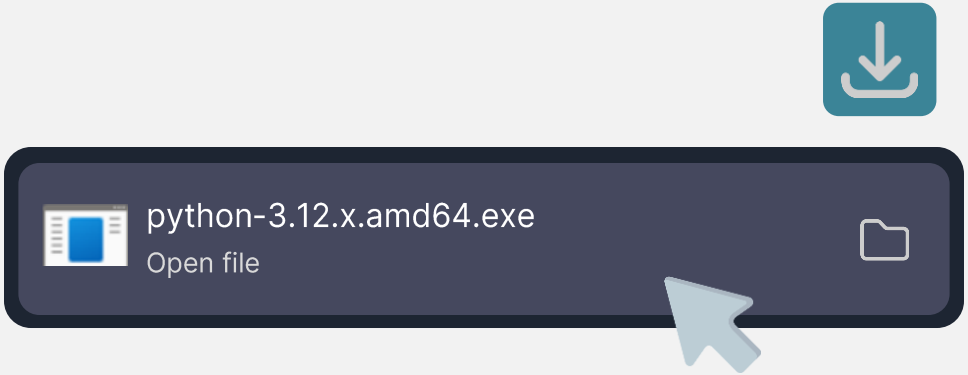
# Environment Setup

## #1

**NOTE:** The latest Python versions may not support older Operating Systems.

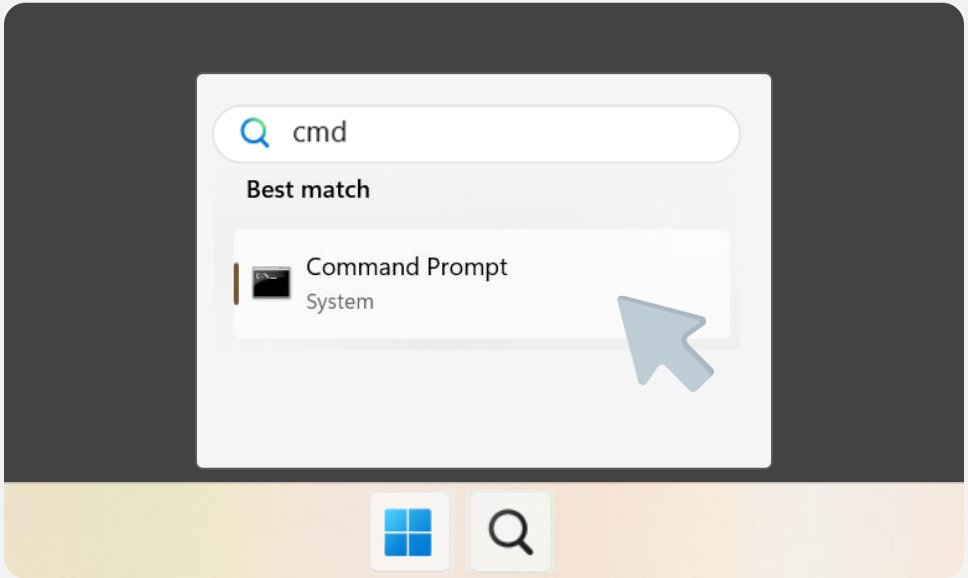
# Environment Setup

**#2**



# Environment Setup

## #3




**NOTE:** Command Prompt is pre-installed on Windows.

# Environment Setup

#4

Command Prompt

```
Microsoft Windows [Version 10.0.22631.4037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\juan>python -V
```

# Namespace

A virtual container containing all known identifiers.

**NOTE:** You can imagine the the desk as the **namespace**, anything outside is unknown and not usable.

# Source Code

A program in textual form.

**NOTE**: A source code is any valid program written in any programming language, such as Python.

# Module

Any Python file are considered as a module.

**NOTE**: A single module is like a puzzle piece that you can assemble into a larger program.

# Main Program

The Python file to be executed in command line.

**NOTE**: The main program is like the **first page** of a document, this is the entry point of the whole sets of programs.

# Package

A folder containing `__init__.py` file.

**Packages**

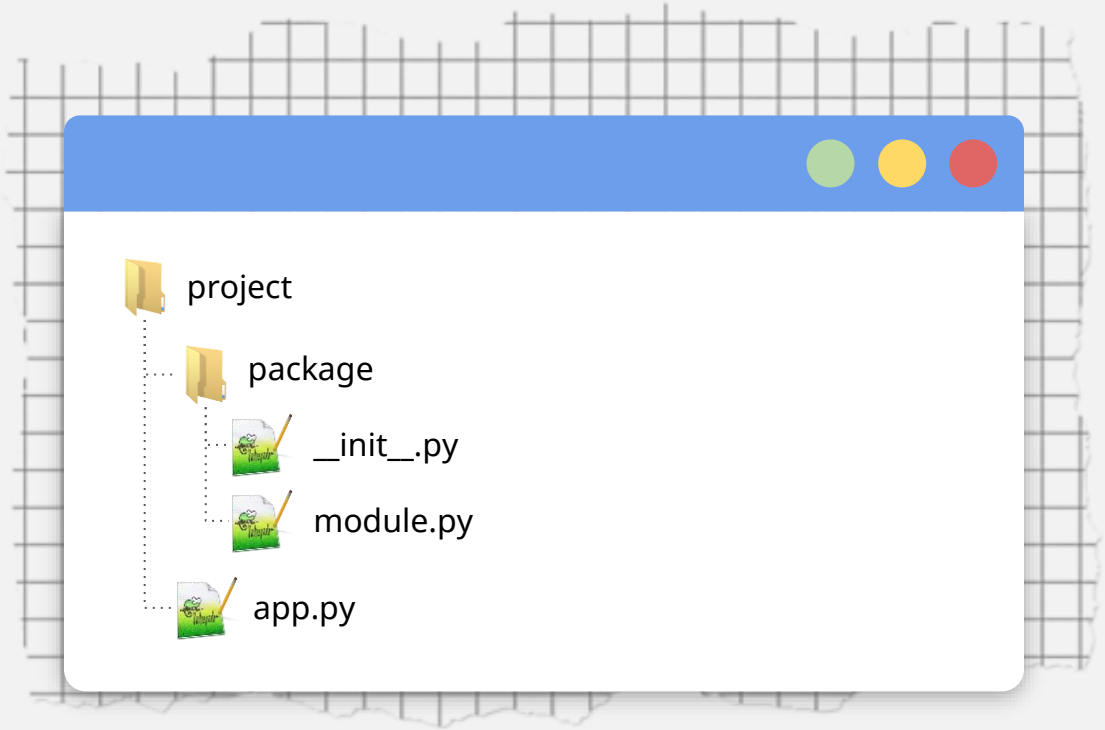
**NOTE**: A package is a folder containing a set of common Python modules or programs.

# Library

A folder containing related packages.

**NOTE**: Therefore, a **software** is a bundle of multiple files, programs, packages, and libraries that makes it whole.

# Project Repository



# __init__.py

__init__.py

```
1
2
3
4
5
6
7
8
9
```

**NOTE**: The **__init__.py** file can be empty, or set to initialize certain imports and variable initializations.

# Import Keyword

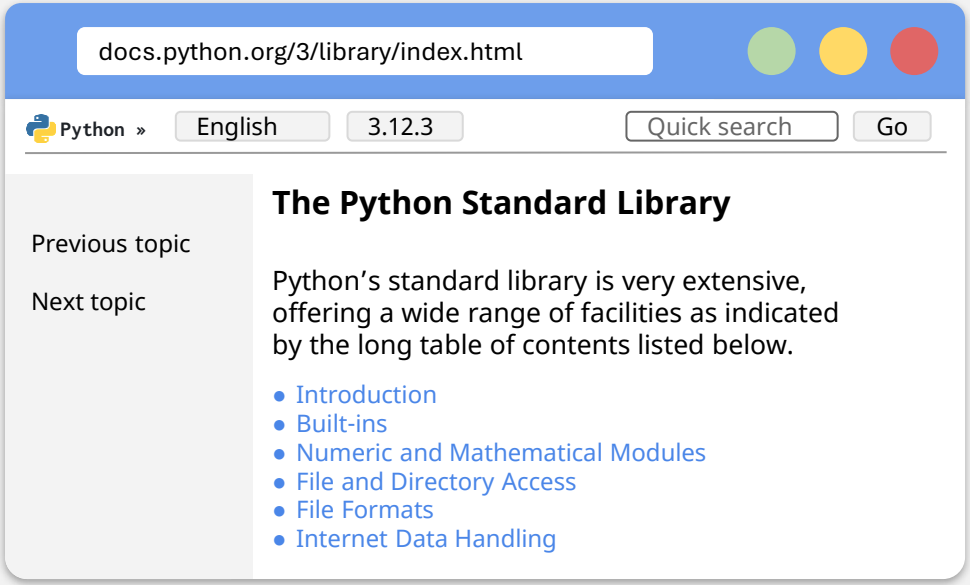
```
1  import library
2  from package import module
3  from package.module import function
```

Back-end Web Dev

# Package Manager

# Standard Library

The installation of Python allows programmers to use **specialized modules** to perform specific operations instead of doing it from scratch.

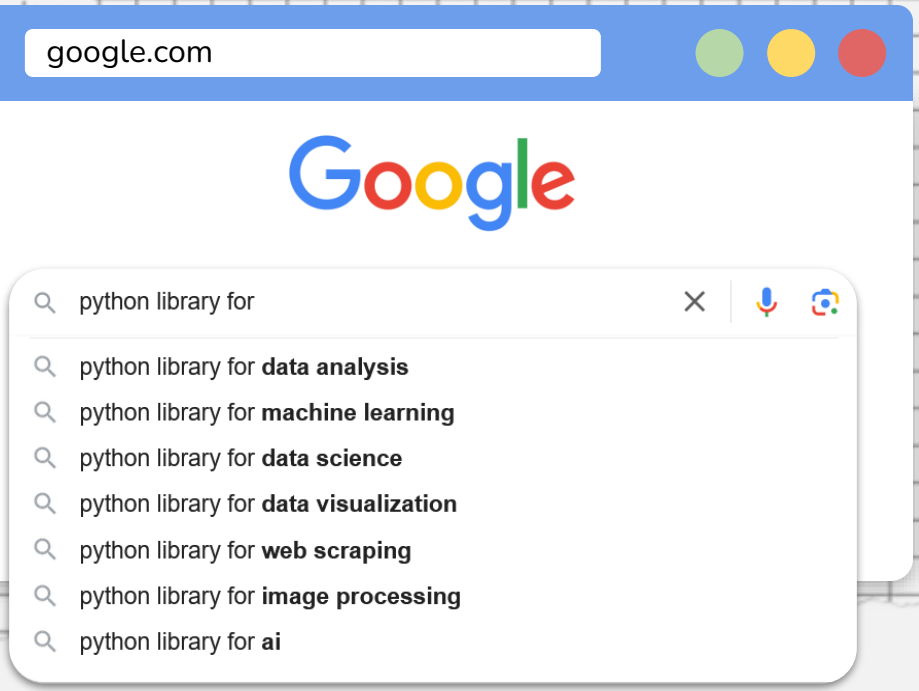



**NOTE:** Also includes built-in functions, types, and exceptions.

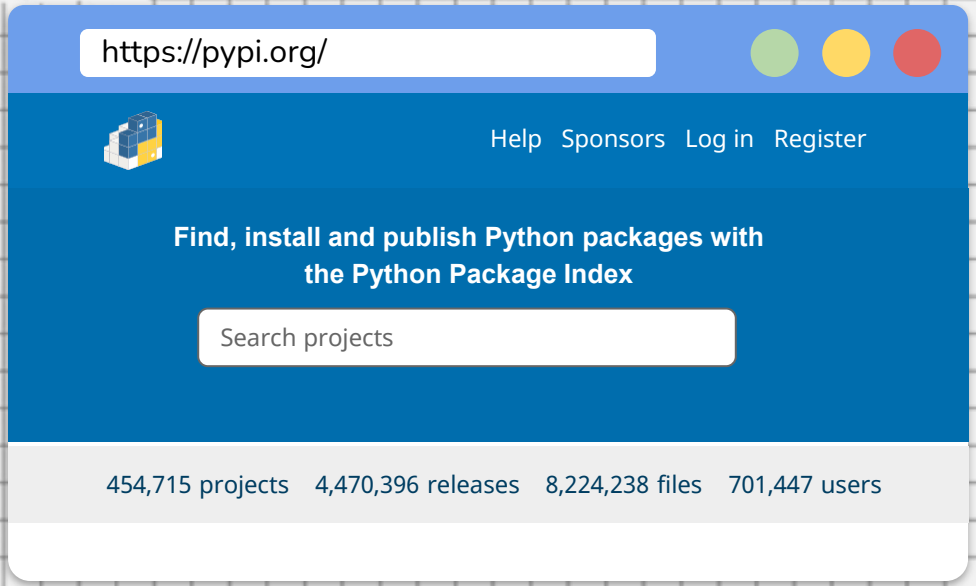
# What is a Package Manager?

It is a tool that simplifies the **installation**, **upgrade**, **removal**, and **management** of dependencies of the installed third-party libraries.

# What is a Package Manager?



# What is a Package Manager?



**NOTE:** The Python Package Index (PyPI) is a repository of software for the Python programming language.

# What is pip?

It the default **Python Package Manager** used to install, uninstall, or update the third-party libraries.

# How to use pip?

#1

Command Prompt

```
C:\project>pip install package
```

Install a new package (or library)...

# How to use pip?

## #2

Command Prompt

```
C:\project>pip install -U package
```

Update a package...

# How to use pip?

## #3

Command Prompt

```
C:\project>pip uninstall package
```

Remove an installed package...

# How to use pip?

## #4

Command Prompt

```
C:\project>pip freeze > requirements.txt
```

Save the list of installed packages...

# How to use pip?

## #5

Command Prompt

```
C:\project>pip install -r requirements.txt
```

Re-install all required packages...

Back-end Web Dev

# Third-Party Libraries

# What are Third-Party Libraries?

It is a set of readily downloadable packages that are used to **extend the capabilities of Python**.

# Virtual Environments

*virtualenv* package

It creates an isolated Python project and installs packages within that project only.

* Ensures that a project does not affect other projects and vice versa.

# Virtual Environments

virtualenv.pypa.io/en/latest/

virtualenv

Search

Installation
User Guide
CLI interface
Extend functionality
Development
Release History


```
# No user data
ethicalads:
  topic: devs
  region: global
  type: image
```

**EthicalAds** Reach backend, frontend, DataSci, or DevOps engineers with a content-targeted network

Ad by EthicalAds


v: latest

## Virtualenv

pypi v20.23.0 | implementation cpython | pypy | python 3 | docs passing | chat 76 online | downloads 63M/month
license MIT | issues 79 open | pull requests 0 open | stars 4.5k

`virtualenv` is a tool to create isolated Python environments. Since Python `3.3`, a subset of it has been integrated into the standard library under the venv module. The `venv` module does not offer all features of this library, to name just a few more prominent:

- is slower (by not having the `app-data` seed method),
- is not as extendable,
- cannot create virtual environments for arbitrarily installed python versions (and automatically discover these),
- is not upgrade-able via pip,
- does not

The basic pr
Imagine you
version `2.`
`python3.8`)

Or more ger
change in its
install packa
host python

In all these c

ON THIS PAGE
Useful links

Command Prompt

```
c:/project>pip install virtualenv
```

# Virtual Environments

## #1

Command Prompt

```
C:\project>virtualenv venv
```

**NOTE:** Supply a name that is simple to remember.

# Virtual Environments

## #2

Command Prompt

```
c:/project> .\venv\Scripts\activate

(venv) c:/project>
```

**NOTE:** Activate the virtual environment every time you are working on your project. This specific command is for Windows PCs only, look for the specific command for usage in Unix/Mac.

# Virtual Environments

## #3

Command Prompt

```
(venv) c:/project> deactivate

c:/project>
```

**NOTE:** If you're done coding, deactivate your virtual environment.

# Web Frameworks

*flask* package

It is a lightweight Web Server Gateway Interface (WSGI, WIZ-gee) web application framework, simply a mediator to a webserver.

# Web Frameworks

https://flask.palletsprojects.com/

Project Links

Donate
PyPI Releases
Source Code
Issue Tracker
Chat

Contents

Welcome to Flask
User's Guide
API Reference
Additional Notes

Quick search

Go

filestack
One API to manage all your file uploads. Effortless integration; Reliable performance

Flask

Welcome to Flask's documentation. Flask is a lightweight WSGI web application framework. It is designed to make getting started quick and easy, with the ability to scale up to complex applications.

Get started with Installation and then get an overview with the Quickstart. There is also a more detailed Tutorial that shows how to create a small but complete application with Flask. Common patterns are described in the Patterns for Flask section. The rest of the docs describe each component of Flask in detail, with a full reference in the API section.

Command Prompt

```
c:/project>pip install flask
```

# HTTP Requests

*requests* package

It is a simple HTTP library widely used for web scraping, API integration, and **general-purpose HTTP requests**.

# HTTP Requests

requests.readthedocs.io/en/latest/

Fork me on GitHub

Requests
http got humans

Star 49,538

Requests is an elegant and simple HTTP library for Python, built for human beings.

Useful Links

Quickstart
Advanced Usage
API Reference
Release History
Contributors Guide

Recommended Packages and Extensions

## Requests: HTTP for Humans™

Release v2.30.0. (Installation)

downloads/month 301M | license Apache 2.0 | wheel yes | python 3.7 | 3.8 | 3.9 | 3.10 | 3.11

**Requests** is an elegant and simple HTTP library for Python, built for human beings.

**Behold, the power of Requests:**

```
>>> r = requests.get('https://api.github.com/user', auth=('user', 'pass'))
>>> r.status_code
200
>>> r.headers['content-type']
```

Command Prompt

```
c:/project>pip install requests
```

# Summary

- **Python** is an interpreted programming language.
- **Python Standard Library** is like a toolbox to help programmers ideate and write programs faster.
- **pip** simplifies the management of third-party libraries.
- **Python Third-Party Libraries** are packages aimed to solve specialized areas in programming such as web development.

Scan me!

# Evaluation Form

forms.gle/TncdC5nEKJZYucVz6

# Thank You!

# References


- Bauer, F. L. (1972). Software Engineering. In Information Processing (p. 71).
- IEEE. (1990). IEEE Standard Glossary of Software Engineering Terminology. IEEE Std 610.12-1990, 1–84. IEEE Std 610.12-1990. https://doi.org/10.1109/IEEESTD.1990.101064
- https://www.researchgate.net/figure/Centralized-decentralized-and-distributed-systems
- https://developer.mozilla.org/en-US/docs/Web/HTTP